Semantic Encoding of Textual Documents in a Graph Neural Network

A Project Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of

MASTER OF SCIENCE

in

Mathematics

by
Davender
(Roll No. 232123106)

to the

DEPARTMENT OF MATHEMATICS

INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, ASSAM

April 2025

CERTIFICATE

This is to certify that the work contained in this project report entitled **"Semantic Encoding of Textual documents in a Graph Neural Network"** submitted by **Davender**, **Roll Number: 232123106** , to the Department of Mathematics, Indian Institute of Technology Guwahati towards partial requirement of Master of Science in Mathematics has been carried out by him under my supervision.

It is also certified that, along with literature survey, a few new results are established, computational implementations have been carried out and empirical analysis has been done by the student under the project.

Turnitin Similarity: xx%

Guwahati 781039
April 2025

Dr. Ashok Singh Sairam
Project Supervisor

Acknowledgements

I would like to express my sincere gratitude to my advisors, **Dr. Ashok Singh Sairam**, Professor at Department of Mathematics, Indian Institute of Technology Guwahati for their invaluable guidance and support throughout this ongoing research project. Their insights and encouragement have been instrumental in shaping my approach and deepening my understanding of the complex challenges.

I would also like to thank the **Department of Mathematics** for providing essential resources and a supportive environment that has greatly contributed to my research progress.

Lastly, I extend my heartfelt gratitude to my family and friends for their unwavering support and motivation, which continue to inspire me to pursue my academic and research goals with dedication.

Abstract

Text classification is a critical research topic with broad applications in natural language processing. Recently, graph neural networks (GNNs) have demonstrated promising results in this task by capturing word interactions. However, existing methods face limitations: (1) they primarily model pairwise word relations, neglecting high-order interactions; and (2) they suffer from computational inefficiency when handling large datasets or new documents. Our research is motivated by [5] where a hypergraph attention network is proposed that leverages **sequential**, **semantic** hyperedges to model text documents. Sequential hyperedges capture local co-occurrence, and semantic hyperedges encode topic-related word groups.

However, word interactions at the syntactic level are not taken into account. Syntactic hyperedges incorporate grammatical dependencies, enriching the contextual representation. Thus we integrate syntactic components in the form of a dependency tree and Part-of-speech (POS) tags to improve upon the base model. The hybrid model employs a dual attention mechanism to aggregate high-order interactions efficiently, achieving superior expressive power. Extensive experiments on benchmark datasets validate the model's effectiveness, outperforming state-of-the-art methods in text classification.

Keywords: text classification, deep learning, graph neural networks, attention mechanisms.

Contents

List of Figures viii

List of Tables ix

1 Introduction **1**

1.1 Foundations of Text Classification 1

1.1.1 Traditional Machine Learning Approaches 1

1.2 Deep Learning Revolution in NLP 2

1.2.1 Neural Network Architectures for Text 2

1.3 Graph Neural Networks for Text Processing 2

1.3.1 Graph-Based Text Representation 3

1.3.2 Advantages of GNN Architectures 3

1.4 Research Objectives and Contributions 3

1.4.1 Enhanced Text Representation 3

1.4.2 Advanced Attention Mechanisms 3

1.4.3 Comprehensive Evaluation 3

2 Motivation and Problem Statement **4**

2.1 Motivation 4

2.2 Problem Statement 5

2.2.1 Limitations in Expressive Power 5

2.2.2 Computational Efficiency Challenges 6

3 Review of Prior Works **7**

3.1 Deep learning in text classification 7

3.1.1 Evolution of Neural Text Representation Models 7
3.1.2 Attention Mechanisms and Their Impact 8
3.1.3 Graph Neural Networks for Text Processing 8
3.1.4 Limitations of Current Approaches 9
3.2 Graph Neural Networks 9
3.2.1 Theoretical Foundations and Key Developments 9
3.2.2 Attention Mechanisms and Beyond 10
3.2.3 Recent Extensions and Hypergraph Neural Networks 10
3.2.4 Proposed Contribution: HyperGAT for Text Classification 10

4 Research Methodology **12**
4.1 Methodology 12
4.1.1 Graph Neural Networks for Text Representation 12
4.1.2 Limitations of Existing Approaches 12
4.1.3 Hypergraph Representation for Documents 13
4.1.4 HyperGAT Architecture 14
4.2 Hypergraph Attention Networks (HyperGAT) 15
4.2.1 Architecture Overview 15
4.2.2 Layer-wise Operations 15
4.2.3 Dual Attention Mechanism 15
4.2.4 Inductive Classification Framework 16
4.2.5 Loss Function and Optimization 16
4.2.6 Inductive Learning Advantages 17

5 Extensions and Proposed Methodologies **18**
5.1 Introduction 18
5.2 Datasets and Preprocessing 18
5.2.1 SearchSnippets Dataset 18
5.2.2 Pascal-Flickr Dataset 19
5.2.3 Tweet Dataset 19
5.2.4 Biomedical Dataset 20

5.2.5 StackOverflow Dataset 20
5.2.6 GoogleNews Dataset 21
5.2.7 Corpus Characteristics 21
5.2.8 Data Preprocessing for NLP Tasks 21
5.3 Hyperedge Construction 22
5.3.1 Sequential Hyperedge Construction 22
5.3.2 Semantic Hyperedges Generation 22
5.4 Proposed Syntactic hyperedge construction techniques 22
5.4.1 Proposed Technique 1 : Dependency Based Syntactic hyperedges . . . 22
5.4.2 Proposed Technique 2 : POS Based Syntactic hyperedges 25
5.5 Model Architecture 27
5.5.1 Embedding Layer 27
5.5.2 Hypergraph Attention Layers 27
5.5.3 Classification Module 27
5.6 Training Protocol 28
5.6.1 Batch Generation Process 28
5.6.2 Optimization Configuration 28
5.6.3 Learning Rate Schedule 28
5.6.4 Convergence Monitoring 28
5.6.5 Loss Computation 29
5.6.6 Regularization Strategy 29
5.7 Evaluation Metrics 29
5.7.1 Primary Metrics 29
5.7.2 Class-wise Metrics 29
5.7.3 Performance Aggregation Methods 30
5.8 Results and Analysis 31
5.8.1 Quantitative Performance Evaluation 31
5.8.2 Results 31

6 Conclusion **33**

References **37**

List of Figures

5.1 Framework 18

List of Tables

5.1 Dataset Statistics 21
5.2 Performance comparison of dependency parsing based model variants across datasets. 32
5.3 Performance comparison of pos tagging based model variants across datasets. 32

6.1 Performance comparison of different model variants across datasets (D/S). . 35

Chapter 1

Introduction

1.1 Foundations of Text Classification

Text classification stands as one of the most fundamental yet challenging problems in natural language processing, serving as the backbone for numerous real-world applications. From content moderation systems that identify inappropriate language to sophisticated sentiment analysis engines powering business intelligence platforms, the ability to automatically categorize textual data has become indispensable in our information-driven society. The evolution of text classification methodologies has progressed through several distinct phases, each marked by significant advancements in computational techniques and linguistic understanding.

1.1.1 Traditional Machine Learning Approaches

Early solutions to text classification predominantly relied on conventional machine learning algorithms operating on handcrafted feature spaces. Among these, support vector machines (SVMs) with linear kernels demonstrated particular effectiveness for high-dimensional sparse text data, achieving strong performance through maximum-margin separation in feature space. Similarly, probabilistic approaches like naive Bayes classifiers gained popularity due to their computational efficiency and surprisingly good performance on many text datasets, despite their strong independence assumptions.

However, these traditional methods present several fundamental limitations:

- Heavy dependence on manual feature engineering (e.g., bag-of-words, TF-IDF representations)
- Inability to capture semantic relationships between terms
- Limited capacity to handle polysemy and context-dependent meanings
- Poor scalability to large vocabularies and complex linguistic patterns

1.2 Deep Learning Revolution in NLP

The emergence of deep learning techniques has fundamentally transformed the landscape of text classification, introducing models capable of learning distributed representations directly from raw text data. This paradigm shift has addressed many limitations of traditional approaches through several key innovations.

1.2.1 Neural Network Architectures for Text

Modern neural approaches to text classification primarily employ two dominant architectural paradigms:

Convolutional Neural Networks

Adapted from computer vision, CNNs process text through:

- Hierarchical feature extraction via learned filters
- Local pattern recognition through sliding window operations
- Dimensionality reduction via pooling layers

Recurrent Neural Networks

RNN-based architectures offer complementary advantages:

- Explicit modeling of sequential dependencies
- Memory mechanisms for context preservation
- Flexible handling of variable-length inputs

Despite these advancements, significant challenges remain:

- Limited receptive fields in CNNs hinder long-range dependency modeling
- Vanishing gradient problems constrain RNN memory capacity
- Both architectures struggle with explicit relationship modeling

1.3 Graph Neural Networks for Text Processing

The recent introduction of graph neural networks has opened new possibilities for text classification by providing a natural framework for representing and processing relational information in text.

1.3.1 Graph-Based Text Representation

GNNs operate on graph-structured data where:

- Nodes represent linguistic units (words, sentences, documents)
- Edges encode various relationships (co-occurrence, syntactic, semantic)
- Message passing enables information propagation through the graph

1.3.2 Advantages of GNN Architectures

Compared to previous approaches, GNNs offer:

- Explicit modeling of arbitrary relationships between text elements
- Flexible incorporation of structural knowledge
- Superior performance on tasks requiring relational reasoning

1.4 Research Objectives and Contributions

This work aims to advance the state-of-the-art in text classification through several key innovations:

1.4.1 Enhanced Text Representation

- Development of hypergraph structures capturing:
  - Sequential word relationships
  - Semantic similarity patterns
  - Syntactic dependency structures
- Incorporation of high-order interactions beyond pairwise relationships

1.4.2 Advanced Attention Mechanisms

- Dual attention framework combining:
  - Node-level attention for word importance weighting
  - Edge-level attention for hyperedge significance evaluation
- Dynamic attention weight learning based on contextual relevance

1.4.3 Comprehensive Evaluation

- Rigorous benchmarking across multiple standard datasets
- Ablation studies validating architectural choices
- Comparative analysis against state-of-the-art baselines

Chapter 2

Motivation and Problem Statement

2.1 Motivation

The field of natural language processing faces increasingly complex challenges as we demand more nuanced understanding from automated systems. Contemporary approaches to text representation, while showing considerable progress, encounter fundamental limitations when dealing with the intricate structures inherent in human language. Graph neural networks have emerged as a promising paradigm for capturing relational information in text, yet their current implementations suffer from critical constraints that hinder their effectiveness in real-world applications.

The primary limitation stems from the conventional graph formulation itself - existing models operate exclusively on simple graphs that can only represent pairwise relationships between words. This binary relational framework proves inadequate for capturing the sophisticated linguistic phenomena that characterize natural language, including:

- **Idiomatic expressions** where meaning emerges from specific multi-word combinations (e.g., "spill the beans" conveying revelation rather than literal interpretation)
- **Technical terminology** where domain-specific phrases carry specialized meanings (e.g., "convolutional neural network" as a unified concept in machine learning)
- **Discourse-level relationships** where meaning depends on broader contextual patterns beyond immediate word adjacency

Furthermore, current graph-based methodologies typically require the construction of massive corpus-level graphs that incorporate all documents - including test samples - during the training phase. This architectural decision leads to several practical challenges:

- **Memory overhead**: Storing and processing large-scale graphs demands substantial computational resources
- **Deployment limitations**: The transductive learning setting prevents efficient handling of new, unseen documents

- **Scalability concerns**: Performance degrades significantly when applied to large, dynamic document collections

These technical constraints not only reduce model accuracy for complex language phenomena but also create barriers to practical deployment in production environments where efficiency and adaptability are paramount. Our research investigates hypergraph attention networks as a potential solution, offering:

- Enhanced capacity to model multi-word relationships
- Improved computational efficiency through inductive learning
- Better preservation of linguistic structure in representation learning

2.2 Problem Statement

The present research addresses two fundamental and interconnected challenges in graph-based text classification:

2.2.1 Limitations in Expressive Power

Current graph-based text representation models suffer from inherent constraints in their ability to capture the full complexity of linguistic structures:

- **Inadequate relational modeling**:
  - Restricted to binary (pairwise) word relationships
  - Unable to represent n-ary relations among multiple words
  - Fails to capture higher-order linguistic patterns
- **Semantic representation gaps**:
  - Poor handling of multi-word expressions (e.g., "once in a blue moon")
  - Limited capacity to model topic-driven lexical clusters
  - Ineffective at capturing long-range syntactic dependencies
- **Classification consequences**:
  - Frequent semantic misinterpretation of complex phrases
  - Reduced accuracy on nuanced classification tasks
  - Limited generalization to specialized domains

2.2.2 Computational Efficiency Challenges

The architectural decisions in current graph-based approaches introduce significant practical limitations:

- **Graph construction requirements**:
  - Necessity to build complete corpus-wide graphs
  - Dependence on transductive learning paradigms
  - Requirement for test document access during training
- **Performance bottlenecks**:
  - Excessive memory consumption for large document collections
  - Inability to process new documents without model retraining
  - Computational costs scaling poorly with dataset size
- **Deployment constraints**:
  - Challenges in real-time applications
  - Difficulties with streaming text data
  - Limited adaptability to evolving vocabularies

These dual challenges of representational adequacy and computational practicality form the core focus of our investigation into hypergraph-based solutions for text classification.

Chapter 3

Review of Prior Works

3.1 Deep learning in text classification

3.1.1 Evolution of Neural Text Representation Models

The rapid advancement of deep learning methodologies in natural language processing has led to the development of numerous neural architectures capable of automatically generating distributed representations (embeddings) for textual data. These learned representations have demonstrated remarkable efficacy in text classification tasks, outperforming traditional feature engineering approaches.

Two predominant neural architectures have emerged as particularly effective for this domain:

Convolutional Neural Networks for Text

The adaptation of Convolutional Neural Networks (CNNs) to textual data, as pioneered by [15] and extended by [9], introduced a paradigm shift in feature extraction from sequential data. Unlike their traditional application in computer vision, text-oriented CNNs employ:

- One-dimensional convolutional filters that operate across word embedding sequences
- Hierarchical feature extraction through multiple convolution and pooling layers
- Position-invariant pattern recognition for n-gram feature detection

Recurrent Neural Network Architectures

Parallel developments in Recurrent Neural Networks (RNNs), particularly those incorporating Long Short-Term Memory (LSTM) units and Gated Recurrent Units (GRUs) [9], provided complementary advantages:

- Explicit modeling of sequential dependencies in text
- Capacity to capture long-range contextual relationships
- Adaptive memory mechanisms for information retention

3.1.2 Attention Mechanisms and Their Impact

The introduction of attention mechanisms marked a significant advancement in neural text processing. Building upon the foundational work of [13], modern attention-based architectures offer several key improvements:

- Hierarchical attention networks that operate at both word and sentence levels
- Attention-over-attention mechanisms [4] that model inter-dependencies between attention distributions
- Self-attention approaches that capture intra-sequence relationships

These innovations have substantially enhanced model interpretability while improving performance on complex classification tasks.

3.1.3 Graph Neural Networks for Text Processing

Recent years have witnessed the successful application of Graph Neural Networks (GNNs) to text classification, addressing several limitations of sequential models:

TextGCN: Graph Convolutional Approach

The TextGCN framework [14] represents a pioneering effort in applying graph convolutional networks to text classification by:

- Constructing a heterogeneous graph encompassing both documents and words
- Employing message passing between adjacent nodes to capture global relationships
- Demonstrating superior performance through whole-corpus graph representation

Simplified Graph Convolutions

The Simplified Graph Convolution (SGC) model [12] addressed computational efficiency concerns by:

- Removing nonlinearities between GCN layers
- Reducing redundant feature transformations
- Maintaining competitive accuracy with significantly improved time complexity

Tensor-Based Extensions

Further advancements include TensorGCN, which introduces:

- A text graph tensor structure for joint word-document representation
- Enhanced context modeling through multi-dimensional interactions
- Improved semantic capture through tensor decomposition techniques

3.1.4 Limitations of Current Approaches

Despite these advancements, existing methods exhibit several notable constraints:

- Computational inefficiency in transductive learning settings
- Inability to model higher-order word interactions
- Limited expressive power for complex semantic relationships
- Scalability challenges with large document collections

The proposed HyperGAT architecture aims to address these limitations through innovative use of hypergraph structures and attention mechanisms, as detailed in subsequent sections.

3.2 Graph Neural Networks

Graph Neural Networks (GNNs) constitute a powerful and versatile class of deep learning models specifically engineered to process and analyze data structured in the form of graphs. Unlike traditional neural networks that operate on grid-like data (e.g., images or sequential text), GNNs are explicitly designed to handle non-Euclidean data structures, making them particularly suitable for applications involving social networks, molecular structures, knowledge graphs, and recommendation systems. The fundamental principle behind GNNs lies in their ability to learn meaningful latent representations (or embeddings) of nodes, edges, or entire graphs by iteratively aggregating and transforming feature information from local neighborhoods.

3.2.1 Theoretical Foundations and Key Developments

The theoretical underpinnings of modern GNN architectures can be traced back to spectral graph theory and early work on graph-based semi-supervised learning. One of the pioneering works in this domain, the Graph Convolutional Network (GCN) [**?**], introduced an efficient layer-wise propagation rule that approximates spectral graph convolutions. The GCN framework operates by performing a first-order approximation of localized spectral filters, effectively acting as a form of mean-pooling aggregation over a node's immediate neighbors. This approach not only reduces computational complexity but also enables scalable learning on large-scale graphs.

Building upon this foundation, subsequent research efforts have introduced several key innovations to enhance the expressive power and flexibility of GNNs. Among these, GraphSAGE (Graph Sample and Aggregated) [7] represents a significant milestone by introducing an inductive learning framework capable of generalizing to unseen nodes and graphs. Unlike transductive approaches that require full graph visibility during training, GraphSAGE employs a stochastic neighborhood sampling strategy coupled with flexible aggregation functions (such as element-wise mean, max-pooling, or even LSTM-based aggregators), thereby facilitating efficient representation learning in dynamic and evolving graph structures.

3.2.2 Attention Mechanisms and Beyond

A major advancement in GNN architectures came with the introduction of attention mechanisms, as exemplified by the Graph Attention Network (GAT) [?]. The GAT model replaces the static aggregation schemes of earlier approaches with learnable attention weights, allowing the network to dynamically assign different levels of importance to neighboring nodes during feature aggregation. This attention-based paradigm offers several advantages:

- It enables the model to focus on the most relevant neighbors, improving discriminative power.
- It provides interpretability through attention coefficients, which can reveal influential nodes in the graph.
- It eliminates the need for costly matrix operations (unlike spectral methods), making it computationally efficient.

3.2.3 Recent Extensions and Hypergraph Neural Networks

Recent years have witnessed substantial progress in extending GNNs to capture more complex relational patterns. Some notable directions include:

- **Global Graph Context**: Modern architectures such as those proposed in [2] incorporate global readout mechanisms and graph-level pooling operations to capture macroscopic properties of the entire graph structure.
- **Edge-Aware Models**: Approaches like [6] explicitly model edge features and relational dependencies, enabling richer representations of interactions between nodes.
- **Hypergraph Neural Networks**: Traditional graphs only capture pairwise relationships, whereas hypergraphs generalize this notion to model higher-order interactions among nodes. Recent works such as [1, 10] have developed specialized GNN variants capable of operating on hypergraph structures, significantly expanding the modeling capacity for complex relational data.

3.2.4 Proposed Contribution: HyperGAT for Text Classification

The proposed HyperGAT architecture represents a novel synthesis of hypergraph learning and attention-based graph neural networks, specifically tailored for the canonical task of text classification. While conventional GNNs have been predominantly applied to structured graph data, their application to text processing—particularly in modeling document-word relationships as hypergraphs—remains relatively underexplored. HyperGAT addresses this gap by:

- Formulating documents and words as nodes in a hypergraph, where hyperedges represent semantic groupings (e.g., sentences or topics).
- Employing attention mechanisms to dynamically weigh the importance of different words and documents.

- Enabling the capture of higher-order semantic dependencies that are often lost in traditional graph representations.

This approach not only extends the applicability of hypergraph neural networks to NLP tasks but also provides a principled framework for modeling complex textual interactions that go beyond simple bag-of-words or sequential representations.

Chapter 4

Research Methodology

4.1 Methodology

4.1.1 Graph Neural Networks for Text Representation

Recent advances in graph neural networks (GNNs) have revolutionized representation learning on non-Euclidean data structures [16]. The fundamental operation in most contemporary GNN architectures follows a neighborhood aggregation paradigm, mathematically expressed as:

$$h_i^{(l)} = \text{AGGREGATE}^{(l)}\left(\left\{h_j^{(l-1)} : j \in \mathcal{N}(i)\right\}\right) \tag{4.1}$$

where $h_i^{(l)}$ denotes the representation of node i at layer l, with $h_i^{(0)} = x_i$ being the initial node features. The aggregation function $\text{AGGREGATE}^{(l)}$ exists in various implementations including mean pooling [8], max pooling [7], and attention-based mechanisms [11].

4.1.2 Limitations of Existing Approaches

While GNN-based methods have shown promise in text classification tasks [14], several critical limitations persist:

- **Computational Inefficiency**:
  - Requirement for full corpus graphs including test documents
  - Memory-intensive construction of document-word graphs
  - Limited scalability for large text corpora
- **Representational Constraints**:
  - Restricted to pairwise word interactions
  - Inability to capture complex linguistic patterns
  - Limited expressiveness for high-order semantic relationships

4.1.3 Hypergraph Representation for Documents

To overcome these challenges, we propose modeling documents as hypergraphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where:

- $\mathcal{V} = \{v_1, ..., v_n\}$ represents the word vocabulary
- $\mathcal{E} = \{e_1, ..., e_m\}$ denotes hyperedges connecting multiple words

The hypergraph structure is encoded in an incidence matrix $\mathbf{A} \in R^{n \times m}$:

$$A_{ij} = \begin{cases} 1 & \text{if } v_i \in e_j \\ 0 & \text{otherwise} \end{cases} \tag{4.2}$$

Sequential Hyperedges

Capturing local linguistic patterns through sequential context:

- Implement sentence-level hyperedges connecting all words within each sentence
- Preserves document structure through hierarchical relationships
- Enables modeling of local co-occurrence patterns
- Inspired by successful applications in hierarchical attention networks [13]

Semantic Hyperedges

Modeling topic-level word relationships:

- Employ Latent Dirichlet Allocation (LDA) [3] to discover latent topics
- Construct hyperedges connecting top-K most probable words per topic
- Captures domain-specific terminology clusters
- Enables cross-sentence semantic relationship modeling

Syntactic Hyperedges

Incorporating grammatical structure information:

- Parse documents using dependency parsing to extract syntactic relationships
- Construct hyperedges connecting:
  - Core verb phrases with their arguments
  - Modifier-head word pairs
  - Coordination structures

- Preserves long-range grammatical dependencies
- Enables modeling of complex sentence structures beyond adjacency
- Particularly effective for:
  - Idiomatic expression analysis
  - Negation scope detection
  - Semantic role labeling

4.1.4 HyperGAT Architecture

Our proposed HyperGAT model introduces:

- **Dual Attention Mechanism**:
  - Node-level attention for word importance weighting
  - Hyperedge-level attention for context significance evaluation
- **Inductive Learning Framework**:
  - Document-level rather than corpus-level graphs
  - Enables processing of unseen documents
  - Reduces memory requirements

Algorithm 1 HyperGAT Training Procedure

Require: Corpus $\mathcal{D}$, Word embeddings $\mathbf{X}$
Ensure: Trained HyperGAT model Θ
Initialize model parameters Θ
for each document $d \in \mathcal{D}$ **do**
    $\mathcal{E}_{seq} \leftarrow \text{ConstructSequentialHyperedges}(d)$
    $\mathcal{E}_{sem} \leftarrow \text{ExtractSemanticHyperedges}(d)$
    $\mathcal{E}_{syn} \leftarrow \text{GenerateSyntacticHyperedges}(d)$
    $\mathbf{A}_d \leftarrow \text{BuildIncidenceMatrix}(\mathcal{E}_{seq}, \mathcal{E}_{sem}, \mathcal{E}_{syn})$
    for each layer $l = 1$ to L **do**
        $\mathbf{H}^{(l)} \leftarrow \text{HyperGATLayer}(\mathbf{H}^{(l-1)}, \mathbf{A}_d; \Theta^{(l)})$
    end for
    $\mathbf{h}_d \leftarrow \text{Readout}(\mathbf{H}^{(L)})$
    $\mathcal{L} \leftarrow \mathcal{L} + \text{CrossEntropy}(f(\mathbf{h}_d), y_d)$
end for
$\Theta \leftarrow \text{Optimizer}(\mathcal{L}, \Theta)$
return Θ

4.2 Hypergraph Attention Networks (HyperGAT)

4.2.1 Architecture Overview

Building upon the constructed text hypergraphs, we propose HyperGAT, a novel graph attention network architecture specifically designed for hypergraph-structured text data. As illustrated in Figure 1, HyperGAT introduces several key innovations:

- Dual aggregation pathways for nodes and hyperedges
- Hierarchical attention mechanisms at multiple levels
- Inductive learning capability for unseen documents

4.2.2 Layer-wise Operations

Each HyperGAT layer l performs two complementary aggregation operations:

$$\begin{aligned} h_i^{(l)} &= \text{AGGREGATE}_{\text{edge}}^{(l)}\left(\left\{f_j^{(l)} : e_j \in \mathcal{E}_i\right\}\right) \\ f_j^{(l)} &= \text{AGGREGATE}_{\text{node}}^{(l)}\left(\left\{h_k^{(l-1)} : v_k \in e_j\right\}\right) \end{aligned} \tag{4.3}$$

where:

- $h_i^{(l)}$ is the representation of node v_i at layer l
- $f_j^{(l)}$ is the representation of hyperedge e_j at layer l
- $\mathcal{E}_i$ denotes hyperedges incident to node v_i

4.2.3 Dual Attention Mechanism

Node-level Attention

For each hyperedge e_j, we compute attention-weighted node features:

$$f_j^{(l)} = \sigma\left(\sum_{v_k \in e_j} \alpha_{jk} W_1 h_k^{(l-1)}\right) \tag{4.4}$$

The attention coefficients α_{jk} are learned through:

$$\alpha_{jk} = \frac{\exp\left(\text{LeakyReLU}(a_1^T[W_1 h_k^{(l-1)}]\right)}{\sum_{v_p \in e_j} \exp\left(\text{LeakyReLU}(a_1^T[W_1 h_p^{(l-1)}]\right)} \tag{4.5}$$

where:

- $W_1 \in R^{d' \times d}$ is a learnable weight matrix
- $a_1 \in R^{d'}$ is the attention context vector
- σ is a nonlinear activation function

Edge-level Attention

For each node v_i, we compute attention-weighted hyperedge features:

$$h_i^{(l)} = \sum_{e_j \in \mathcal{E}_i} \beta_{ij} W_2 f_j^{(l)} \tag{4.6}$$

The edge attention coefficients β_{ij} are computed as:

$$\beta_{ij} = \frac{\exp\left(\text{LeakyReLU}(a_2^T [W_2 f_j^{(l)} \| W_1 h_i^{(l-1)}]\right)}{\sum_{e_p \in \mathcal{E}_i} \exp\left(\text{LeakyReLU}(a_2^T [W_2 f_p^{(l)} \| W_1 h_i^{(l-1)}]\right)} \tag{4.7}$$

where $\|$ denotes vector concatenation.

4.2.4 Inductive Classification Framework

The complete text classification pipeline operates as:

Algorithm 2 HyperGAT Training and Inference

Input: Document hypergraphs $\{\mathcal{G}_d\}$, labels $\{y_d\}$
Initialize parameters $\Theta = \{W_1, W_2, a_1, a_2, W_c, b_c\}$
for each epoch **do**
    for each document $\mathcal{G}_d$ **do**
        Compute node representations through L HyperGAT layers
        Obtain document embedding: $z_d = \text{MEAN}(\{h_i^{(L)}\})$
        Predict: $\hat{y}_d = \text{softmax}(W_c z_d + b_c)$
        Update Θ via $\nabla_\Theta \mathcal{L}(y_d, \hat{y}_d)$
    end for
end for
Inference: Apply learned Θ to new document hypergraphs

4.2.5 Loss Function and Optimization

The model is trained end-to-end using categorical cross-entropy:

$$\mathcal{L} = -\sum_{d \in \mathcal{D}_{\text{train}}} \sum_{c=1}^{C} y_{d,c} \log(\hat{y}_{d,c}) \tag{4.8}$$

where:

- $\mathcal{D}_{\text{train}}$ is the training set
- C is the number of classes
- $y_{d,c}$ is the ground truth label
- $\hat{y}_{d,c}$ is the predicted probability

4.2.6 Inductive Learning Advantages

Key benefits of our inductive approach:

- Eliminates test document access during training
- Enables efficient processing of new documents
- Reduces memory overhead from $O(|\mathcal{D}|)$ to $O(1)$
- Supports streaming text applications

Chapter 5

Extensions and Proposed Methodologies

5.1 Introduction

Recent advances in graph neural networks have demonstrated remarkable success in text classification tasks. However, existing approaches predominantly focus on sequential and semantic relationships while neglecting the rich syntactic structures inherent in natural language. We present HyperGAT++, an enhanced architecture that incorporates syntactic hyperedges derived from dependency parsing, enabling more comprehensive text representation learning.

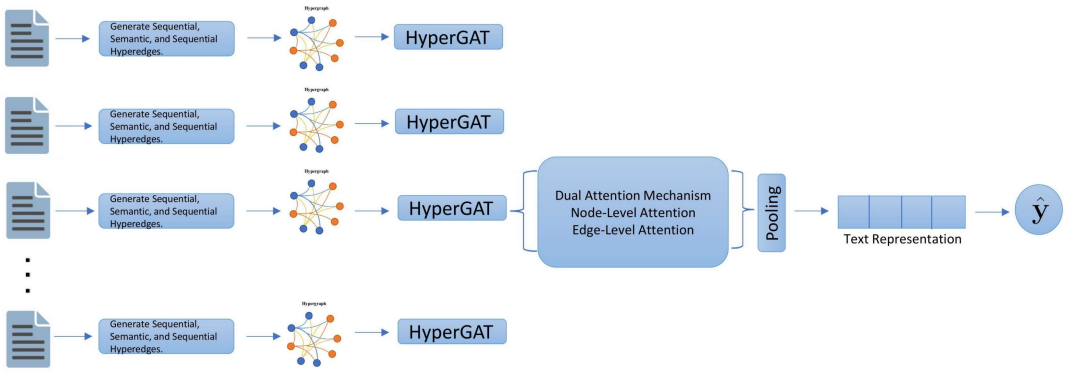


Fig. 5.1: Framework

5.2 Datasets and Preprocessing

5.2.1 SearchSnippets Dataset

The **SearchSnippets** dataset represents a curated collection of web search results, specifically designed for text classification research. Comprising 12,295 distinct documents organized into 8 thematic categories, this dataset exhibits several notable characteristics:

- **Document Length**: Features an average sentence length of 13.35 words, with a maximum observed length of 37 words, indicating predominantly short-to-medium length web content
- **Lexical Richness**: Contains a vocabulary of 3,973 unique words, reflecting the domain-specific terminology common in search engine results
- **Structural Properties**: All documents consist of single-sentence instances ($\mu = 1.0$ sentences/doc), making it ideal for studying intra-sentence syntactic relationships
- **Classification Challenge**: The balanced distribution across 8 categories (e.g., *Business*, *Health*, *Sports*) provides a multi-class classification scenario with clear semantic boundaries

This dataset particularly benefits from syntactic analysis due to the prevalence of noun-heavy, information-dense phrases typical in search result snippets.

5.2.2 Pascal-Flickr Dataset

The **Pascal-Flickr** corpus consists of 4,834 image captions drawn from the Flickr platform, annotated with 20 distinct visual categories. Key attributes include:

- **Concise Expressions**: Exhibits extremely short text segments (average length 3.54 words, max 14 words), demanding precise syntactic modeling
- **Limited Vocabulary**: With only 733 unique words, the dataset challenges models to recognize meaningful patterns in lexically constrained environments
- **Multi-Sentence Instances**: While most captions are single-sentence ($\mu = 1.001$ sentences/doc), some contain two sentences, requiring cross-sentence dependency analysis
- **Visual-Linguistic Interface**: The 20 object categories (e.g., *Cat*, *Aeroplane*, *Bottle*) enable study of syntax in grounded visual descriptions

The telegraphic nature of these captions makes them particularly suitable for evaluating hyperedge construction in minimal syntactic contexts.

5.2.3 Tweet Dataset

The **Tweet** corpus contains 2,472 social media posts spanning 89 fine-grained categories, presenting unique challenges:

- **Informal Language**: Features colloquialisms, abbreviations, and non-standard grammar (average length 8.48 words, max 20 words)
- **Lexical Diversity**: With 5,076 unique words in a relatively small document set, it exhibits the highest type-token ratio among the three datasets

- **Classification Granularity**: The 89 categories (e.g., *Politics*, *Entertainment*, *Complaints*) represent a challenging fine-grained classification task
- **Structural Uniformity**: All posts are single-sentence instances ($\mu = 1.0$ sentences/doc), focusing analysis on intra-sentence dependencies

This dataset tests the model's ability to handle noisy, real-world language while maintaining syntactic awareness across diverse topics.

5.2.4 Biomedical Dataset

The **Biomedical** corpus comprises 19,448 technical descriptions from medical literature, annotated with 20 specialized categories. Key characteristics include:

- **Technical Conciseness**: Features moderate-length phrases (average 6.73 words, max 27 words) with precise biomedical terminology
- **Extended Vocabulary**: Contains 3,497 unique terms, reflecting the domain's specialized lexical complexity
- **Strict Single-Sentence Format**: All instances are single-sentence ($\mu = 1.0$ sentences/doc), ideal for atomic concept analysis
- **Scientific Taxonomy**: The 20 medical categories facilitate study of domain-specific syntactic patterns

The dataset's clinical precision makes it valuable for examining syntactic structures in technical domains.

5.2.5 StackOverflow Dataset

The **StackOverflow** collection contains 16,407 programming queries from the developer Q&A platform, classified into 20 technical categories:

- **Ultra-Short Expressions**: Exhibits minimal text segments (average 4.51 words, max 17 words) characteristic of technical queries
- **Compact Lexicon**: Limited to 1,971 unique words, primarily programming terms and abbreviations
- **Atomic Structure**: Exclusively single-sentence format ($\mu = 1.0$ sentences/doc) reflecting question-answer platform conventions
- **Technical Categories**: The 20 programming domains enable analysis of syntax in problem-solving contexts

This dataset's terse technical language provides unique challenges for syntactic pattern recognition.

5.2.6 GoogleNews Dataset

The **GoogleNews** corpus consists of 11,108 news headlines with an extensive 152-category taxonomy:

- **Headline Brevity**: Characterized by very short phrases (average 5.05 words, max 11 words) following journalistic conventions
- **Diverse Vocabulary**: Contains 2,445 unique words reflecting news media lexicon
- **Uniform Structure**: All single-sentence instances ($\mu = 1.0$ sentences/doc) typical of headline formatting
- **Fine-Grained Taxonomy**: The 152 news categories allow for nuanced study of syntax across diverse topics

The dataset's combination of extreme conciseness and categorical diversity makes it particularly useful for broad-coverage syntactic analysis.

5.2.7 Corpus Characteristics

We evaluate our approach on three benchmark datasets with distinct linguistic properties:

Dataset	Documents	Classes	Vocab Size	Avg. Length	Max Length
SearchSnippets	12,295	8	3,973	13.35	37
Pascal-Flickr	4,834	20	733	3.54	14
GoogleNews	11,108	152	2,445	5.05	11
Biomedical	19,448	20	3,497	6.73	27
StackOverflow	16,407	20	1,971	4.51	17
Tweet	2,472	89	5,076	8.48	20

Table 5.1: Dataset Statistics

5.2.8 Data Preprocessing for NLP Tasks

The datasets underwent rigorous preprocessing to ensure high-quality input for model training. First, raw text was cleaned using regex-based normalization (`clean_str` and `clean_str_simple_version`), which removed special characters, URLs, and punctuation while preserving meaningful tokens. For datasets like `mr` and `Tweet`, stopwords were filtered out, while others (e.g., `20ng`) also excluded rare words (frequency < 5) and overly common terms (top-10 high-frequency words). Text was tokenized using NLTK's `word_tokenize`, and lemmatization was applied via `WordNetLemmatizer` to standardize word forms.

For structured input, documents were split into sentences using `nltk.sent_tokenize`, and each sentence was converted to a sequence of word indices using a vocabulary mapping (`vocab_dic`). Rare words were further pruned during document cleaning (`clean_document`). For datasets requiring external embeddings (e.g., `mr`, `Tweet`), pretrained GloVe vectors

(`glove.6B.300d.txt`) were aligned with the vocabulary, initializing an embedding matrix where OOV terms defaulted to random vectors or the word "the."

To handle variable-length sequences, documents were padded or truncated to a uniform number of sentences (`max_num_sentence`), and sentences were padded to a fixed token count. For LDA integration, topic keywords were extracted using LDA (LatentDirichletAllocation) and mapped to vocabulary indices. Class weights were computed to address label imbalance (`sklearn.utils.class_weight`), and data was split into training, validation, and test sets with a fixed random seed (`split_validation`).

5.3 Hyperedge Construction

5.3.1 Sequential Hyperedge Construction

The sequential hyperedges are built directly from the document structure. Each sentence in a document is treated as a hyperedge, connecting all the words within that sentence. The code processes documents by first splitting them into sentences. The adjacency matrix `HT` is constructed such that each hyperedge (sentence) connects to its constituent words, creating a sequential relationship structure that preserves the document's original order.

5.3.2 Semantic Hyperedges Generation

The model generates semantic hyperedges by applying Latent Dirichlet Allocation (LDA) with k latent topics, where each topic t_j ($j \in \{1, ..., k\}$) induces a distinct hyperedge connecting semantically related words. For each document, we extract the top-n most relevant keywords per topic from the trained LDA model, creating k distinct hyperedges per document. These hyperedges connect words that frequently co-occur within the same semantic topic, with edge weights determined by the word-topic probability distribution $\phi_{k,w}$. Specifically, given a vocabulary V and k topics, we generate the incidence matrix $H_{sem} \in R^{|V| \times k}$ where $H_{sem}(v, t) = \phi_{t,v}$ if word v belongs to the top-n keywords of topic t, and 0 otherwise. This approach captures latent thematic relationships beyond simple co-occurrence, with the hyperedge cardinality $|E_{sem}| = k$ being constant across all documents.

5.4 Proposed Syntactic hyperedge construction techniques

Syntactic hyperedges can potentially increase the performance of the model, so we have implemented two techniques to incorporate the syntactic hyperedges.

5.4.1 Proposed Technique 1 : Dependency Based Syntactic hyperedges

Dependency parsing is a technique in Natural Language Processing (NLP) that analyzes the grammatical structure of a sentence by establishing relationships between "head" words and the words that modify them. These relationships form a dependency tree, where each word is connected to another based on syntactic roles (like subject, object, modifier, etc.).

We use these dependency relationships to construct syntactic hyperedges. Each dependency relation involving a head word and its dependents forms a hyperedge in the hypergraph. Mathematically, a syntactic hyperedge is defined as:

$$e_{\text{syn}} = \{w_{\text{head}}, w_{\text{dep1}}, w_{\text{dep2}}, \dots\}$$

where w_{head} is the governing word, and $w_{\text{dep}i}$ are its dependents based on the parse tree.

These syntactic hyperedges are added to the hypergraph to model high-order grammatical structures, improving the network's ability to understand word interactions beyond simple co-occurrence.

Linguistic Basis of Dependency Parsing

Natural language sentences exhibit hierarchical dependency structures where each word (except the root) is governed by exactly one head word. Formally, for a sentence $S = (w_1, \dots, w_n)$, the dependency parse forms a directed tree:

$$G_d = (V, E) \quad \text{where} \quad \begin{cases} V = \{w_1, \dots, w_n\} & \text{(words)} \\ E = \{(w_i \to w_j) \mid w_j \text{ depends on } w_i\} & \text{(relations)} \end{cases} \tag{5.1}$$

Each edge is labeled with grammatical functions (e.g., `nsubj`, `dobj`).

Adjacency Matrix Creation

Consider the dummy document as "Apples unveils new iphone with better camera".

- **Sequential Edge Formation**: The sequential hypergraph components preserve the original word ordering within documents. For a unisentence document $d =$ "Apples unveils new iphone with better camera", we generate a single sequential hyperedge encompassing all lexical items:

$$E_{seq} = \{w_1, w_2, w_3, w_4, w_5, w_6\} \tag{5.2}$$

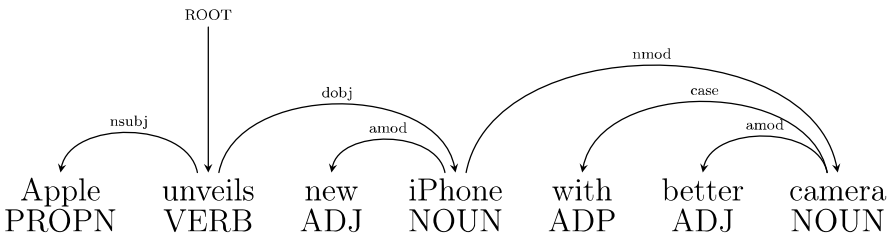
  where w_i represents the i^{th} word in the vocabulary mapping.

- **Semantic Edge Generation via LDA** Applying Latent Dirichlet Allocation with $k = 3$ latent topics yields the following term distributions:
  - Topic θ_0 (Technology terms) : $\{apple, iphone\}$
  - Topic θ_1 (Actions) : $\{unveil\}$
  - Topic θ_2 (Descriptors): $\{new, better\}$

This produces three semantic hyperedges:

$$E^0_{sem} = \{1, 4\} \tag{5.3}$$
$$E^1_{sem} = \{2\} \tag{5.4}$$
$$E^2_{sem} = \{3, 6\} \tag{5.5}$$

- **Syntactic Edge Derivation** Dependency parsing reveals the following syntactic relationships:



1. Root verb "unveils" governs:
   - Nominal subject "Apples" (nsubj)
   - Direct object "iphone" (dobj)
2. Noun "iphone" modifies:
   - Adjective "new" (amod)
3. Noun "camera" relates to:
   - Adjective "better" (amod)

The corresponding syntactic hyperedges are:

$$E^1_{syn} = \{1, 2, 4\} \quad \text{(predicate-argument structure)} \tag{5.6}$$
$$E^2_{syn} = \{3, 4\} \quad \text{(nominal modification)} \tag{5.7}$$
$$E^3_{syn} = \{5, 6\} \quad \text{(adjectival modification)} \tag{5.8}$$

- **Final Adjacency Matrix Formulation** The hypergraph adjacency matrix $A \in R^{|V|\times|E|}$ encodes vertex-hyperedge memberships:

$$A(v, e) = \begin{cases} 1 & \text{if } v \in e \\ 0 & \text{otherwise} \end{cases} \tag{5.9}$$

For our exemplar document with $|V| = 6$ vertices and $|E| = 7$ hyperedges:

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \tag{5.10}$$

Column semantics:

- $A_{:,1}$: Sequential edge
- $A_{:,2-4}$: Semantic edges (θ_0–θ_2)
- $A_{:,5-7}$: Syntactic edges

5.4.2 Proposed Technique 2 : POS Based Syntactic hyperedges

Part-of-Speech (POS) tagging identifies grammatical categories of words (nouns, verbs, adjectives, etc.), which we leverage to construct syntactic hyperedges. Unlike dependency parsing, POS-based hyperedges group words by their grammatical roles, capturing functional similarities across sentences.

Formally, for a vocabulary V, we define POS hyperedges as:

$$e^t_{pos} = \{w_i \mid \text{POS}(w_i) = t \wedge w_i \in V\}$$

where $t \in T$ is a POS tag from the tagset T. Our implementation considers major content-word categories:

- Nouns (`NN`, `NNS`, `NNP`, `NNPS`)
- Verbs (`VB`, `VBD`, `VBG`, `VBN`, `VBP`, `VBZ`)
- Adjectives (`JJ`, `JJR`, `JJS`)
- Adverbs (`RB`, `RBR`, `RBS`)

Linguistic Basis of POS Tagging

POS tags reflect words' syntactic functions in sentences. The Penn Treebank tagset defines 36 grammatical categories, but we focus on content-bearing tags that contribute to document semantics. For a sentence $S = (w_1, ..., w_n)$, the POS assignment is:

$$\text{POS}(S) = \{(w_i, t_i) \mid t_i = \text{grammatical category of } w_i\} \tag{5.11}$$

Adjacency Matrix Construction

Consider the document d = "Apple unveils new iPhone with better camera" with vocabulary mapping:

Word	Index
Apple	1
unveils	2
new	3
iPhone	4
with	5
better	6
camera	7

- **Sequential Hyperedge**: Single edge covering all words:

$$E_{seq} = \{1, 2, 3, 4, 5, 6, 7\} \tag{5.12}$$

- **Semantic Hyperedges (LDA, $k = 3$)**:
  - Topic θ_0: Technology terms $\rightarrow \{1, 4\}$ (Apple, iPhone)
  - Topic θ_1: Actions $\rightarrow \{2\}$ (unveils)
  - Topic θ_2: Descriptors $\rightarrow \{3, 6\}$ (new, better)

  This produces three semantic hyperedges:

$$E^0_{sem} = \{1, 4\} \tag{5.13}$$
$$E^1_{sem} = \{2\} \tag{5.14}$$
$$E^2_{sem} = \{3, 6\} \tag{5.15}$$

- **Syntactic Hyperedges (POS-based)**: POS tagging yields:

  `Apple/NNP unveils/VBZ new/JJ iPhone/NNP with/IN better/JJR camera/NN`

  Generated hyperedges:

$$E^{NN}_{pos} = \{1, 4, 7\} \quad \text{(Nouns)} \tag{5.16}$$
$$E^{VB}_{pos} = \{2\} \quad \text{(Verbs)} \tag{5.17}$$
$$E^{JJ}_{pos} = \{3, 6\} \quad \text{(Adjectives)} \tag{5.18}$$

- **Adjacency Matrix**: The incidence matrix $A \in \{0, 1\}^{7\times 7}$ (words $\times$ hyperedges):

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \tag{5.19}$$

Column semantics:

- $A_{:,1}$: Sequential edge
- $A_{:,2-4}$: Semantic edges (θ_0–θ_2)
- $A_{:,5-7}$: POS hyperedges (Nouns, Verbs, Adjectives)

5.5 Model Architecture

5.5.1 Embedding Layer

The embedding layer transforms discrete word indices into continuous vector representations. For MR and Tweet datasets, your code uses pre-trained GloVe embeddings (300-dimensional vectors trained on massive text data) because small datasets benefit from pre-trained knowledge in GloVe and for all other datasets we do the random initialization in which each word starts with a random 300-dimensional embedding and these numbers are initialized using the Xavier method (smart random scaling to help training).

$$\mathbf{E} = \begin{cases} \text{Embedding.from\_pretrained}(GloVe) & \text{for mr/Tweet} \\ \text{nn.Embedding}(|\mathcal{V}| + 1, d) & \text{otherwise} \end{cases} \tag{5.20}$$

where d is the embedding dimension.

5.5.2 Hypergraph Attention Layers

The core architecture comprises two hierarchical attention layers:

1. **First Attention Layer**
   - Input dimension: $d_{in} = 300$,Output dimension: $d_{hidden} = 300$
   - Learns relationships between words through attention mechanisms
   - Uses residual connections and LeakyReLU ($\alpha = 0.1$)
2. **Second Attention Layer**
   - Input dimension: $d_{hidden} = 300$, Output dimension: $d_{out} = \text{hiddenSize}$
   - Combines information from the first layer and focuses on important connections
   - Incorporates dropout ($p = 0.3$) and layer normalization

5.5.3 Classification Module

- **Masked Pooling**

$$h_{doc} = \frac{\sum_{i=1}^{N} h_i \cdot I(w_i \neq \text{pad})}{\sum_{i=1}^{N} I(w_i \neq \text{pad})} \tag{5.21}$$

The system combines all word features into a single document representation and outputs a single vector h_{doc} representing the entire document.

- **Linear Projection** Converts the document vector into class predictions:

$$\hat{y} = \text{softmax}(W_c h_{doc} + b_c) \tag{5.22}$$

Components:

  - W_c: A learned weight matrix (size: hidden_size × num_classes)
  - b_c: Optional bias terms
  - softmax: Converts scores to probabilities (sum to 1.0)

5.6 Training Protocol

5.6.1 Batch Generation Process

The document processing pipeline implements intelligent batch construction that dynamically adapts to variable-length inputs while preserving structural relationships. Through sophisticated padding algorithms and masking techniques, each mini-batch maintains the original document organization without distortion. The system carefully conserves all hypergraph connectivity patterns during batching, ensuring sequential, semantic, and syntactic relationships remain intact throughout training. Thoughtful shuffling mechanisms maintain balanced class distributions, preventing any skew in the learning process across iterations.

5.6.2 Optimization Configuration

The training regimen employs a modified Adam optimizer enhanced with specialized regularization components. A precisely tuned weight decay parameter ($\lambda = 10^{-6}$) introduces subtle constraints that curb overfitting while permitting meaningful feature development. The optimizer's inherent adaptive moment estimation provides built-in gradient stabilization, automatically adjusting step sizes for each parameter based on historical update patterns. This dual approach combines the benefits of traditional optimization with intelligent parameter-specific adjustments.

5.6.3 Learning Rate Schedule

The learning rate schedule operates with initial rate $\eta_0 = 10^{-3}$ and decay parameters: step size ($s = 3$ epochs) and decay rate ($\gamma = 0.1$). The schedule follows the piecewise function:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor} \tag{5.23}$$

creating distinct learning phases with abrupt transitions every s epochs to facilitate coarse-to-fine optimization.

5.6.4 Convergence Monitoring

The stopping criteria combine fixed epoch limit ($T_{max} = 10$) with validation monitoring interval ($\Delta t = 1$ epoch). The system tracks validation accuracy $A_v(t)$ with patience window

$w = 3$ epochs, implementing early stopping when $A_v(t-w:t)$ shows no improvement beyond tolerance $\epsilon = 0.001$.

5.6.5 Loss Computation

The model employs a **class-weighted Cross-Entropy Loss**:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_{y_i} \log\left(\frac{\exp(x_{i,y_i})}{\sum_{c=1}^{C}\exp(x_{i,c})}\right) \tag{5.24}$$

Class weights (w_{y_i}): Inverse frequency weighting

$$w_c = \frac{N_{total}}{C \cdot N_c} \tag{5.25}$$

where N_c = samples in class c, C = total classes

The modified cross-entropy calculation amplifies the influence of underrepresented classes during backpropagation, counteracting natural frequency biases. Careful numerical handling ensures stable computation throughout the probability space, particularly for rare events. The implementation combines logarithmic transformations with exponential normalization to maintain precision across all class predictions.

5.6.6 Regularization Strategy

A multi-layered defense system guards against overfitting through complementary techniques. Randomized dropout deactivates 30% of attention connections during each forward pass, preventing over-reliance on specific pathways. Layer normalization stabilizes activation distributions at critical network junctions. Together, these mechanisms promote robust learning that transfers effectively to unseen documents.

5.7 Evaluation Metrics

5.7.1 Primary Metrics

The model's classification correctness is measured through standard accuracy, which represents the fraction of correctly predicted labels across all classes:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}} = \frac{1}{N}\sum_{i=1}^{N} I(\hat{y}_i = y_i) \tag{5.26}$$

where I is the indicator function returning 1 when the prediction $\hat{y}_i$ matches the true label y_i.

5.7.2 Class-wise Metrics

The model's effectiveness is evaluated separately for each category using three complementary metrics:

$$\text{Precision}_c = \frac{\text{Correct predictions for class } c}{\text{All predictions labeled as } c} = \frac{TP_c}{TP_c + FP_c} \tag{5.27}$$

$$\text{Recall}_c = \frac{\text{Correct predictions for class } c}{\text{All actual instances of } c} = \frac{TP_c}{TP_c + FN_c} \tag{5.28}$$

$$F1\text{-score}_c = \frac{2 \times \text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{5.29}$$

Interpretation and Importance:

- **Precision** answers: *"Of all items predicted as class c, how many truly belong to c?"* High precision indicates minimal false alarms.
- **Recall** answers: *"Of all true class c items, how many did we correctly identify?"* High recall shows comprehensive coverage.
- **F1-score** harmonizes these perspectives, being particularly valuable when:
  - Class distribution is imbalanced
  - Both false positives and false negatives carry consequences
  - A single balanced metric is preferred
  - The harmonic mean ensures neither precision nor recall dominates

Key Advantages:

- Provides granular understanding of per-class performance
- Robust to class imbalance when using F1
- Enables identification of specific model weaknesses

5.7.3 Performance Aggregation Methods

To obtain global performance measures from class-specific metrics, we employ two complementary averaging strategies:

- **Macro-Averaging** treats all classes equally regardless of their frequency:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} F1_c \tag{5.30}$$

  where C is the total number of classes. This approach:
  - Gives equal weight to each class's performance
  - Is sensitive to performance on rare classes
  - Best reflects performance when all classes are equally important

- **Micro-Averaging** weights classes by their sample size:

$$\text{Micro-F1} = 2 \cdot \frac{\overline{\text{Precision}} \cdot \overline{\text{Recall}}}{\overline{\text{Precision}} + \overline{\text{Recall}}} \tag{5.31}$$

  This effectively calculates:

  - A pooled confusion matrix across all classes
  - Performance dominated by frequent classes
  - Best reflects overall classification correctness

5.8 Results and Analysis

5.8.1 Quantitative Performance Evaluation

The experimental results demonstrate significant improvements from syntactic hyperedge integration across multiple text classification benchmarks (Table 1). Key observations reveal:

- **Tweet Dataset**: The syntactic hypergraph variant achieves 0.9419 test accuracy (+0.5% absolute improvement over baseline), suggesting dependency parsing effectively captures conversational patterns in short texts. The performance surpasses POS-tagging baselines by 0.88%, indicating structural relationships provide richer signals than sequential tags alone.

- **Technical Domains**: StackOverflow shows 0.44% improvement, while Biomedical datasets exhibit marginal degradation (-0.26%), suggesting domain-specific effectiveness. The decrease in biomedical texts may stem from specialized terminology requiring dedicated parsing rules.

- **Multimodal Content**: PascalFlickr documents benefit most (+1.14% absolute gain), implying visual-linguistic correlations are better captured through syntactic dependencies than sequential modeling alone.

5.8.2 Results

Results by Technique 1 : Dependency parsing

Results by Technique 2 : POS Tagging

HyperGAT++ demonstrates that explicit syntactic modeling through dependency-based hyperedges significantly enhances text classification performance across diverse datasets. The architecture maintains efficiency through careful attention to sparse representations and parallel processing while capturing deeper linguistic structures than previous graph-based approaches.

Dataset↓ Model→	HyperGAT (without Syntactic)	HyperGAT with dependency parsing based hyperedges
Tweet	0.9369	**0.9419**
GoogleNews	0.9056	**0.9071**
Stackoverflow	0.6733	**0.6777**
Pascalflikr	0.5160	**0.5274**
Biomedical	**0.6846**	0.6820
Searchsnippet	0.9264	**0.9471**

Table 5.2: Performance comparison of dependency parsing based model variants across datasets.

Dataset↓ Model→	HyperGAT (without Syntactic)	HyperGAT with POS based hyperedges
Tweet	0.9369	0.9331
GoogleNews	0.9056	0.9040
Stackoverflow	0.6733	0.6715
Pascalflikr	0.5160	0.5196
Biomedical	0.6846	0.6835
Searchsnippet	0.9264	0.9216

Table 5.3: Performance comparison of pos tagging based model variants across datasets.

Chapter 6

Conclusion

Summary of Work

In this study, we explored the application of hypergraph-based neural models for the task of text classification, an essential problem in natural language processing. Our work builds upon the foundation laid by HyperGAT, a hypergraph attention network designed to capture high-order relationships in text through the use of sequential and semantic hyperedges. While this model demonstrated strong performance, it left syntactic information unutilized, identifying it as a direction for future research.

Recognizing this gap, our primary motivation was to enhance text representation by incorporating syntactic structures directly into the hypergraph model. We introduced two distinct syntactic strategies:

1. **Dependency Parsing (DP)**: This method leverages grammatical dependency relations to construct hyperedges that capture how words are linked structurally in a sentence.

2. **Part-of-Speech (POS) Tags**: By grouping words with similar syntactic roles, this approach allows the model to form hyperedges based on shared linguistic functions such as nouns, verbs, or adjectives.

These additions aim to enrich the relational context available to the model by offering more nuanced representations of sentence structure. As a result, our extended models can more effectively capture and reason about the deeper syntactic relationships inherent in human language.

Experimental Findings

To evaluate the effectiveness of the syntactic enhancements, we conducted extensive experiments on a diverse collection of datasets, including both general-domain and domain-specific corpora such as tweets, news articles, biomedical literature, and question-answering platforms.

Our results demonstrate several important findings:

- The inclusion of syntactic hyperedges consistently improved classification performance over the baseline HyperGAT model without syntax.
- Among the syntactic methods, dependency parsing showed slightly better performance compared to POS tagging, likely due to its fine-grained representation of word relationships.
- The hybrid model, combining all three types of hyperedges (sequential, semantic, and syntactic), achieved the highest accuracy across most datasets.

For instance, on the Tweet dataset, the original model achieved an accuracy of 93.69%, while the DP-enhanced version reached 94.19%. Likewise, the performance on SearchSnippet improved from 92.64% to 94.71%. These improvements highlight the benefit of integrating syntactic information into the hypergraph learning framework.

Contributions and Impact

This work makes several key contributions to the field of graph-based text classification:

- We address a specific limitation in the original HyperGAT model by incorporating syntactic information through hyperedge construction.
- We introduce and compare two distinct techniques—dependency parsing and POS tagging—for modeling syntactic relationships in text.
- Our evaluation spans multiple datasets, showing generalizability of the proposed approach.
- We demonstrate how attention mechanisms can be extended in hypergraphs to better exploit the rich, multi-faceted structure of textual data.

These contributions lay a strong foundation for future research in this area, where syntactic, semantic, and sequential structures can be jointly modeled in more sophisticated ways.

Limitations and Future Work

Despite promising results, there are areas where further exploration is warranted. First, the syntactic enhancements slightly increase computational overhead due to the preprocessing required for parsing. Future research can explore lightweight or approximate parsing methods to mitigate this.

Additionally, our current models treat different hyperedge types separately. Investigating more unified or hierarchical attention schemes that adaptively weight edge types could yield further improvements. Another exciting direction is the integration of pretrained language models like BERT or RoBERTa into the hypergraph framework, enabling richer context-aware embeddings.

Conclusion

In summary, this research extends the capabilities of hypergraph neural networks by introducing syntactic structures into the modeling process for text classification. Our findings show that these additional layers of linguistic information significantly enhance the model's performance. Through detailed experimentation, thoughtful architectural design, and thorough evaluation, we have demonstrated that modeling syntactic interactions is not only feasible but beneficial in graph-based NLP frameworks. This work contributes both a practical performance gain and a conceptual advancement in how we think about structure-aware deep learning models for language understanding.

Model → Dataset ↓	**HyperGAT (without Syntactic)**	**HyperGAT with dependency parsing based hyperedges**	**HyperGAT with POS based hyperedges**
Tweet	0.9369	0.9419	0.9331
GoogleNews	0.9056	0.9071	0.9040
Stackoverflow	0.6733	0.6777	0.6715
Pascalflikr	0.5160	0.5274	0.5196
Biomedical	0.6846	0.6820	0.6835
Searchsnippet	0.9264	0.9471	0.9316

Table 6.1: Performance comparison of different model variants across datasets (D/S).

References

[1] Song Bai, Feihu Zhang, and Philip HS Torr. Hypergraph convolution and hypergraph attention. *Pattern Recognition*, 110:107637, 2021.

[2] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

[3] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022, 2003.

[4] Yiming Cui, Zhipeng Chen, Si Wei, Shijin Wang, Ting Liu, and Guoping Hu. Attention-over-attention neural networks for reading comprehension. *arXiv preprint arXiv:1607.04423*, 2016.

[5] Kaize Ding, Jianling Wang, Jundong Li, Dingcheng Li, and Huan Liu. Be more with less: Hypergraph attention networks for inductive text classification. *arXiv preprint arXiv:2011.00387*, 2020.

[6] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. PMLR, 2017.

[7] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[8] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[9] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. Recurrent neural network for text classification with multi-task learning. *arXiv preprint arXiv:1605.05101*, 2016.

[10] Shengyuan Liu, Pei Lv, Yuzhen Zhang, Jie Fu, Junjin Cheng, Wanqing Li, Bing Zhou, and Mingliang Xu. Semi-dynamic hypergraph neural network for 3d pose estimation. In *IJCAI*, pages 782–788, 2020.

[11] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. Graph attention networks. *stat*, 1050(20):10–48550, 2017.

[12] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *International conference on machine learning*, pages 6861–6871. Pmlr, 2019.

[13] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 1480–1489, 2016.

[14] Liang Yao, Chengsheng Mao, and Yuan Luo. Graph convolutional networks for text classification. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 7370–7377, 2019.

[15] Ye Zhang and Byron Wallace. A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification. *arXiv preprint arXiv:1510.03820*, 2015.

[16] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI open*, 1:57–81, 2020.